LAPORAN PRAKTIKUM
PENGUJIAN PERANGKAT LUNAK
PENGUJIAN NON-FUNCTIONAL
PERFORMANCE TESTING MENGGUNAKAN APACHE JMETER

Disusun oleh:

Anggota Kelompok	: Hafidz Rizqullah Prasetya
	: Januarsyah Akbar
	: Matthew Hayunaji Priantara
	: Akmal Manggala Putra
Kelas	: PL4B2
Dosen Pengampu	: Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

Daftar Isi

Daftar Isi . . . . . **ii**

Daftar Gambar . . . . . **iii**

Tujuan Praktikum **1**

Dasar Teori **2**
2.1 Pengujian Non-Fungsional (*Non-Functional Testing*) 2
2.2 Apache JMeter 3

Hasil dan Pembahasan **4**
3.1 Penjelasan Tools: Apache JMeter 4
3.1.1 Fitur Utama (*Features*) 4
3.1.2 Metrik Pengujian (*Metrics*) 4
3.1.3 Instalasi (*Installation*) 5
3.1.4 Cara Kerja (*How it Works*) 6
3.2 Hasil Pengujian 6
3.2.1 Langkah-langkah Pengujian 6
3.2.2 Konfigurasi Pengujian 9
3.2.3 Hasil dan Analisis Performa 10
3.2.3.1 *Response Time* 10
3.2.3.2 *Throughput* dan Kecepatan Transaksi 11
3.2.3.3 *Network Load* (Beban Jaringan) 11
3.2.3.4 *Error Rate* (Tingkat Kegagalan) 11

Kesimpulan **12**

Daftar Pustaka **13**

Daftar Gambar

1 Klasifikasi Pengujian Non-Fungsional 3

1 Menambahkan Thread Group 7
2 Konfigurasi Thread Group 7
3 Menambahkan HTTP Request 7
4 Konfigurasi HTTP Request 8
5 Menambahkan View Results Tree 8
6 Menambahkan Summary Report 9
7 Menyimpan dan Menjalankan Test 9
8 Laporan Hasil Pengujian (Summary Report) 10
9 Sampel *Response* Sukses pada View Results Tree 10
10 Sampel *Response* Gagal pada View Results Tree 11

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dan tujuan pengujian non-fungsional (*non-functional testing*), khususnya *performance testing* dan *load testing*.
2. Mahasiswa mampu menjelaskan fitur, metrik, cara instalasi, dan cara kerja tools *Apache JMeter*.
3. Mahasiswa mampu melakukan pengujian performa pada suatu website dan menganalisis hasilnya berdasarkan metrik yang relevan seperti *Response Time*, *Throughput*, *Transactions per Second (TPS)*, dan *Error Rate*.

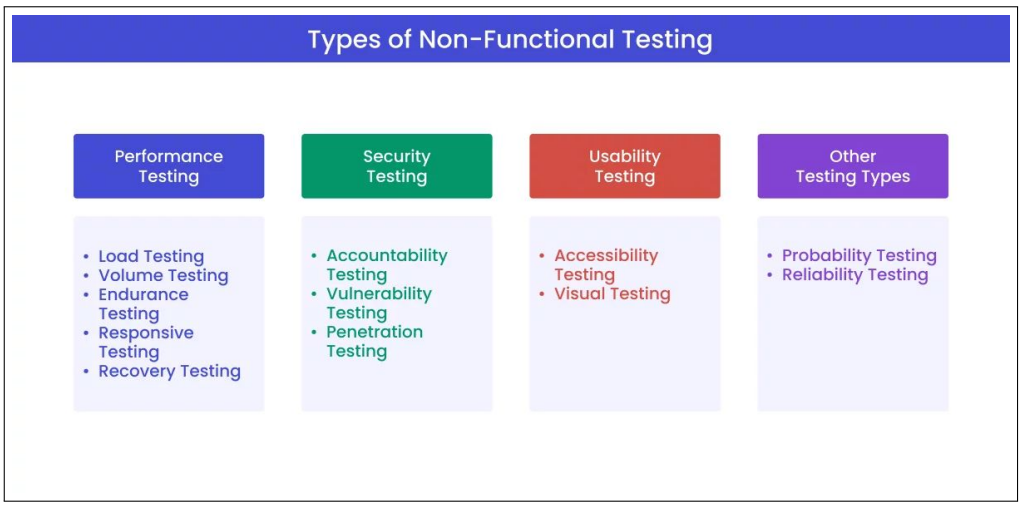
Dasar Teori

2.1 Pengujian Non-Fungsional (*Non-Functional Testing*)

Pengujian non-fungsional adalah jenis pengujian perangkat lunak yang berfokus pada *bagaimana* suatu sistem bekerja, bukan *apa* yang sistem lakukan. Berbeda dengan pengujian fungsional yang memverifikasi fitur dan perilaku sistem terhadap spesifikasi, pengujian non-fungsional mengukur aspek-aspek kualitas seperti performa, skalabilitas, keandalan, dan keamanan.

Beberapa jenis pengujian non-fungsional yang umum dilakukan antara lain:

- ***Performance Testing*:** Mengukur kecepatan, responsivitas, dan stabilitas sistem di bawah beban kerja tertentu.
- ***Load Testing*:** Menguji perilaku sistem di bawah kondisi beban yang diantisipasi, untuk memastikan sistem dapat menangani jumlah pengguna yang diharapkan.
- ***Stress Testing*:** Menguji sistem melebihi kapasitas normalnya untuk menemukan titik kegagalan.
- ***Scalability Testing*:** Mengukur kemampuan sistem untuk meningkatkan kapasitas secara vertikal maupun horizontal.
- ***Security Testing*:** Mengidentifikasi celah keamanan dan kerentanan pada sistem [3].



Gambar 1: Klasifikasi Pengujian Non-Fungsional

2.2 Apache JMeter

Apache JMeter adalah aplikasi sumber terbuka (*open-source*) berbasis Java yang dikembangkan oleh Apache Software Foundation. JMeter dirancang khusus untuk melakukan *load testing* dan mengukur performa berbagai jenis layanan. JMeter awalnya dikembangkan untuk menguji aplikasi web, namun kini mendukung berbagai protokol seperti HTTP/HTTPS, FTP, JDBC, LDAP, JMS, SMTP, dan POP3.

Sebagai sebuah tools *performance testing*, JMeter bekerja dengan cara mensimulasikan banyak pengguna virtual yang mengakses sistem secara bersamaan. Hasil pengujian kemudian dikumpulkan dan dianalisis untuk mengetahui kapasitas, bottleneck, dan keandalan sistem yang diuji [1].

Hasil dan Pembahasan

3.1 Penjelasan Tools: Apache JMeter

3.1.1 Fitur Utama (*Features*)

Apache JMeter memiliki sejumlah fitur unggulan yang menjadikannya salah satu tools *performance testing* paling populer:

- **Dukungan Multi-Protokol:** JMeter mendukung berbagai protokol pengujian, di antaranya HTTP/HTTPS, FTP, Database via JDBC, LDAP, SOAP/REST, JMS, serta SMTP dan POP3 untuk email.
- **GUI dan Non-GUI Mode:** JMeter dilengkapi antarmuka grafis (GUI) yang intuitif untuk pembuatan skenario uji, serta mode CLI (*Command Line Interface*) yang direkomendasikan untuk eksekusi pengujian guna menghemat memori.
- **Perekaman Skenario:** Fitur *HTTP(S) Test Script Recorder* memungkinkan pengguna merekam aktivitas browsing secara langsung dan mengubahnya menjadi skenario pengujian secara otomatis.
- **Parameterisasi Data (*Data-Driven Testing*):** JMeter mendukung penggunaan file CSV sebagai sumber data dinamis melalui komponen *CSV Data Set Config*, sehingga pengujian dapat menggunakan data yang berbeda-beda pada setiap iterasi.
- **Laporan Dinamis (*HTML Dashboard Report*):** Setelah pengujian selesai, JMeter dapat menghasilkan laporan HTML yang kaya visualisasi, mencakup grafik performa, statistik ringkas, dan analisis distribusi waktu respons.
- **Ekstensibilitas melalui Plugin:** Ekosistem plugin JMeter sangat kaya, memungkinkan penambahan jenis sampler, listener, dan visualisasi baru sesuai kebutuhan spesifik pengujian.

3.1.2 Metrik Pengujian (*Metrics*)

Berikut adalah metrik-metrik utama yang diukur dan dianalisis dalam laporan hasil pengujian JMeter:

- **Response Time (Waktu Respons):** Waktu total yang dibutuhkan server sejak menerima permintaan hingga mengirimkan seluruh respons kembali ke klien. Nilai ini biasanya ditampilkan dalam milidetik (ms). Semakin rendah nilainya, semakin baik performa server.

- **Throughput:** Jumlah permintaan (*requests*) yang berhasil diproses oleh server per satuan waktu, biasanya dinyatakan dalam *requests/second*. Metrik ini mengukur kapasitas total server.
- **Transactions per Second (TPS):** Jumlah transaksi penuh (satu siklus lengkap permintaan-respons yang berhasil) yang diselesaikan per detik. Berbeda dengan *throughput* yang menghitung semua permintaan, TPS biasanya hanya menghitung transaksi yang sukses.
- **Error Rate (Tingkat Kesalahan):** Persentase permintaan yang gagal atau menerima respons kesalahan (kode HTTP 4xx atau 5xx) dari total seluruh permintaan yang dikirim. Idealnya bernilai 0%.
- **Connect Time:** Waktu yang dibutuhkan klien untuk membangun koneksi jaringan awal (TCP Handshake) dengan server, sebelum data mulai dikirimkan.
- **Latency:** Waktu dari saat permintaan dikirim hingga byte pertama respons diterima dari server. Berbeda dengan *Response Time* yang mengukur hingga byte terakhir.

3.1.3 Instalasi (*Installation*)

Berikut adalah langkah-langkah untuk menginstal dan menjalankan Apache JMeter:

1. **Prasyarat – Instalasi Java:** JMeter berbasis Java dan membutuhkan *Java Runtime Environment* (JRE) atau *Java Development Kit* (JDK) versi 8 atau lebih baru. Verifikasi instalasi Java dengan menjalankan perintah berikut di terminal:

   ```
   java -version
   ```

2. **Unduh Apache JMeter:** Kunjungi halaman unduhan resmi JMeter di `https://jmeter.apache.org/download_jmeter.cgi`. Unduh file biner terbaru dalam format `.zip` (Windows) atau `.tgz` (macOS/Linux).
3. **Ekstrak File:** Ekstrak file yang telah diunduh ke direktori pilihan Anda. Misalnya: `C:\JMeter\` di Windows atau `/opt/jmeter/` di Linux.
4. **Jalankan JMeter:**
   - **Windows:** Navigasikan ke folder `bin` di dalam direktori ekstraksi, lalu jalankan file `jmeter.bat`.
   - **macOS / Linux:** Buka terminal, navigasikan ke folder `bin`, lalu jalankan perintah:

     ```
     ./jmeter
     ```

   Antarmuka grafis (GUI) JMeter akan terbuka dan siap digunakan.

3.1.4 Cara Kerja (*How it Works*)

JMeter bekerja dengan mensimulasikan sejumlah pengguna virtual yang mengirimkan permintaan ke server target secara bersamaan. Komponen-komponen utama dalam sebuah rencana pengujian (*Test Plan*) JMeter adalah sebagai berikut:

1. **Test Plan:** Wadah utama yang menampung seluruh komponen pengujian.
2. **Thread Group:** Komponen inti yang merepresentasikan pengguna virtual. Di sini dikonfigurasi jumlah pengguna (*Number of Threads*), periode percepatan (*Ramp-Up Period*), dan jumlah pengulangan (*Loop Count*).
3. **Sampler:** Mendefinisikan jenis permintaan yang akan dikirim ke server (misalnya *HTTP Request* untuk menguji website).
4. **Config Elements:** Komponen konfigurasi seperti *HTTP Header Manager* dan *CSV Data Set Config* untuk menyetel detail permintaan.
5. **Listeners:** Komponen yang menangkap dan menampilkan hasil pengujian, seperti *View Results Tree*, *Summary Report*, dan *Aggregate Report*.

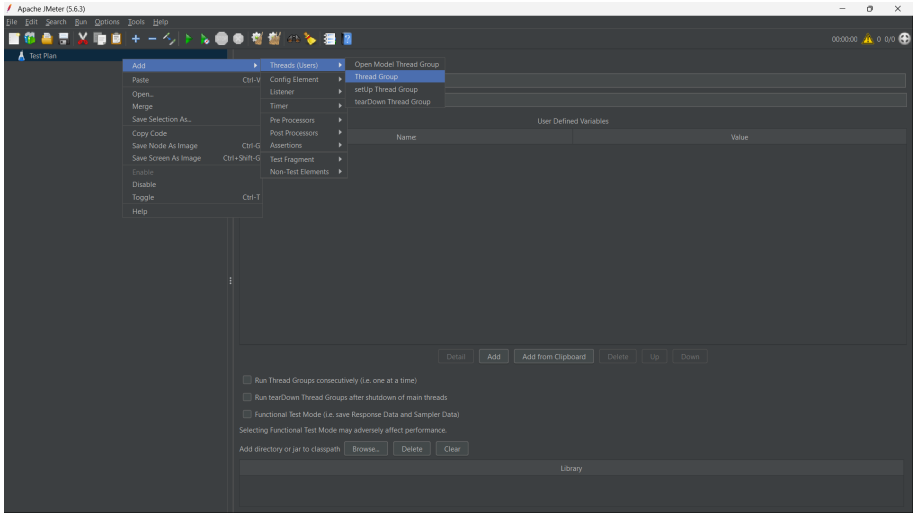
Alur kerja utama JMeter adalah: **(1)** Test Plan dibuat dan dikonfigurasi melalui GUI. **(2)** Pengujian dieksekusi (disarankan via CLI untuk pengujian beban besar). **(3)** JMeter mengumpulkan seluruh data respons. **(4)** Laporan HTML dihasilkan dan dianalisis oleh tim.

3.2 Hasil Pengujian

3.2.1 Langkah-langkah Pengujian

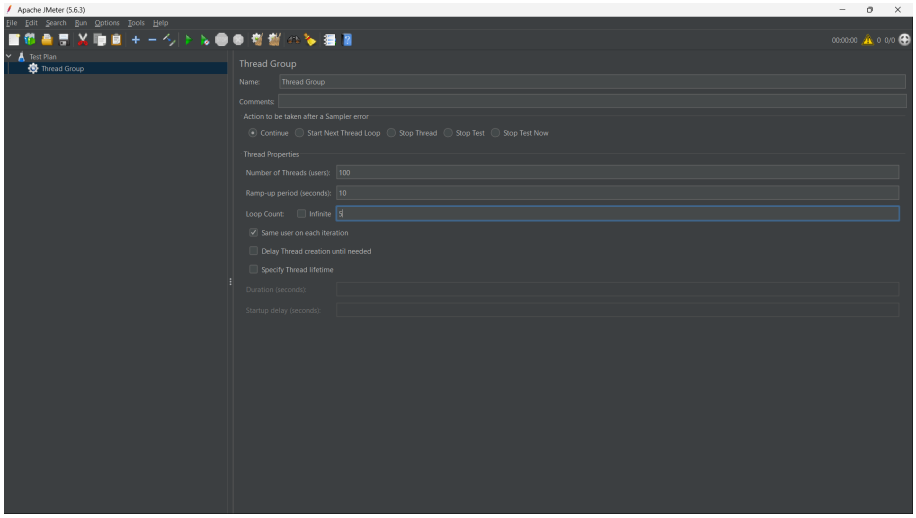
Berikut adalah langkah-langkah sistematis yang dilakukan untuk menyimulasikan beban pengunjung pada website target:

1. **Membuka JMeter:** Menjalankan file `jmeter.bat` (di Windows) atau `./jmeter` (di Mac/Linux) dari dalam folder `bin`. Setelah terbuka, akan terlihat sebuah **Test Plan** kosong di panel sebelah kiri.
2. **Menambahkan Pengguna Virtual (Thread Group):**
   - Klik kanan pada tulisan **Test Plan** di panel kiri, lalu pilih **Add** > **Threads (Users)** > **Thread Group**.



Gambar 1: Menambahkan Thread Group

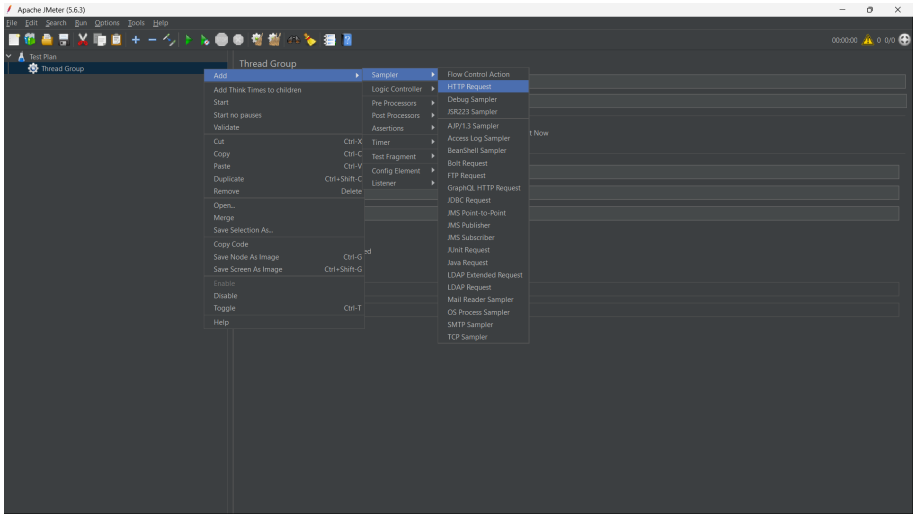
- Di panel kanan, atur konfigurasi jumlah pengguna simulasi.



Gambar 2: Konfigurasi Thread Group

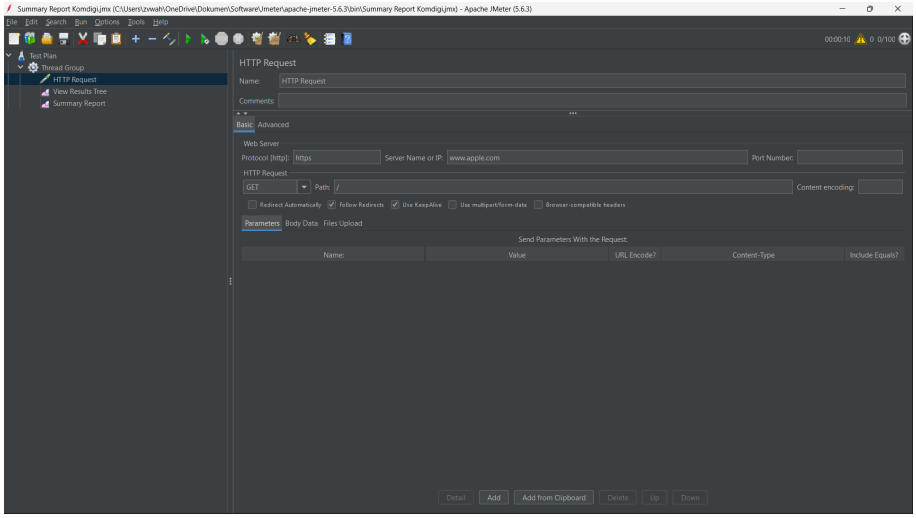
3. **Menentukan Target Website (HTTP Request):**

   - Klik kanan pada **Thread Group** yang baru dibuat, lalu pilih **Add** > **Sampler** > **HTTP Request**.



Gambar 3: Menambahkan HTTP Request

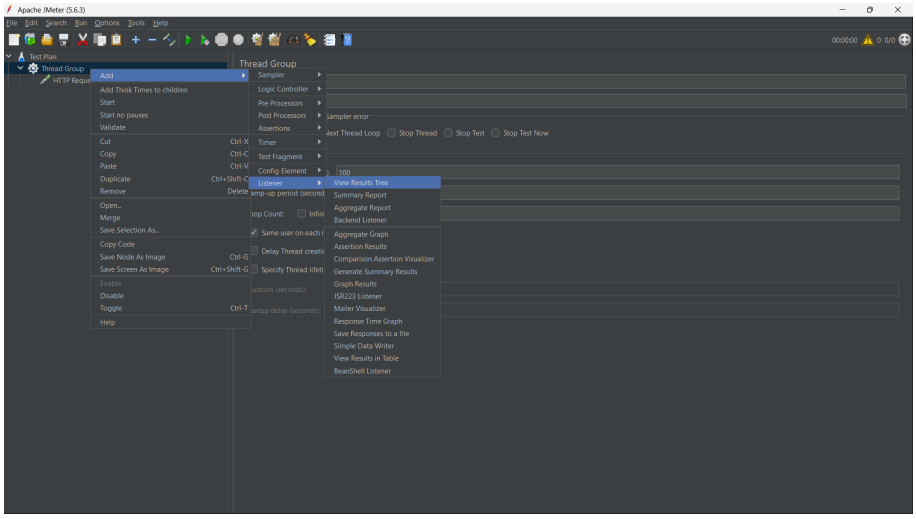
- Menentukan protokol (misal: `https`), nama domain server, dan *path* URL spesifik yang akan diuji.



Gambar 4: Konfigurasi HTTP Request

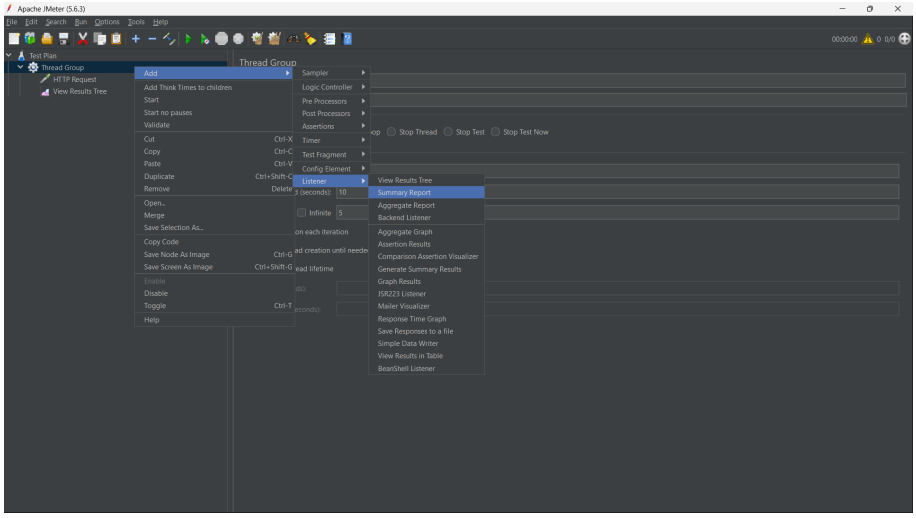
4. **Menambahkan Alat Pemantau Hasil (Listener):**

- Klik kanan pada **Thread Group** lalu pilih **Add** > **Listener** > **View Results Tree** (untuk melacak *request/response* per item).



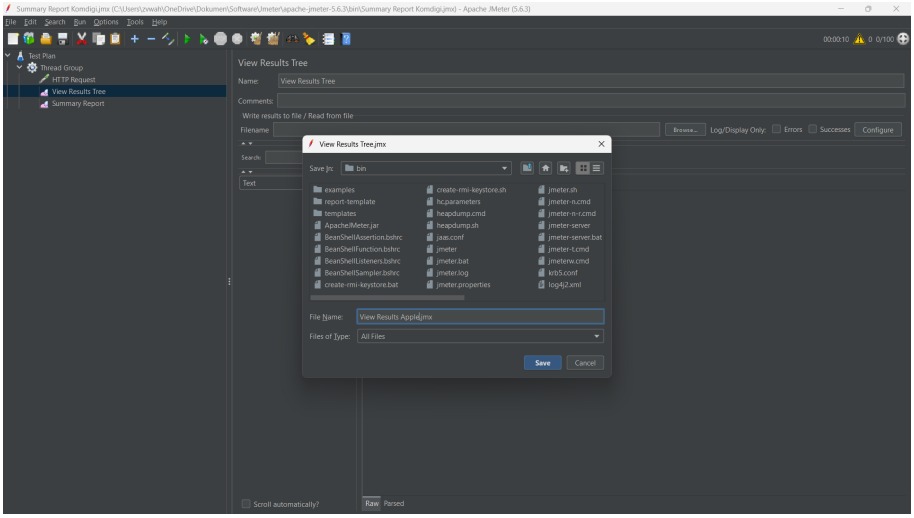
Gambar 5: Menambahkan View Results Tree

- Ulangi dan tambahkan **Add** > **Listener** > **Summary Report** (untuk melihat tabulasi rekap metrik performa).



Gambar 6: Menambahkan Summary Report

5. **Menjalankan Pengujian (Run):** Menyimpan proyek (*save*) dan mengklik tombol **Start** (ikon segitiga hjau) untuk memulai pengetesan aliran data asinkron.



Gambar 7: Menyimpan dan Menjalankan Test

6. **Mengambil Data untuk Laporan:** Setelah indikator selesai, tabel pada **Summary Report** dapat diekspor atau dicek nilainya (seperti *Average Response Time*, *Error %*, dan *Throughput*) untuk dianalisis.

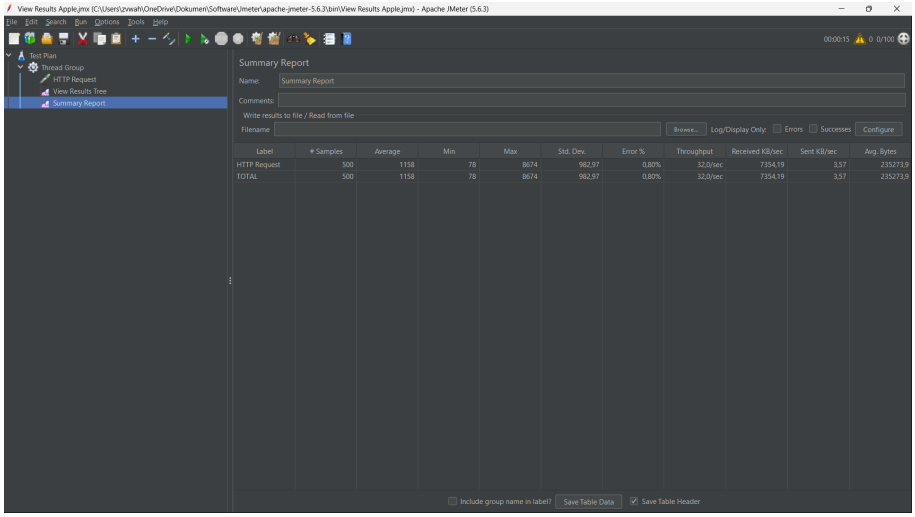
3.2.2 Konfigurasi Pengujian

Pengujian kali ini difokuskan pada halaman utama website Apple (`https://www.apple.com`). Modifikasi parameter pada *Thread Group* disetel sebagai berikut:

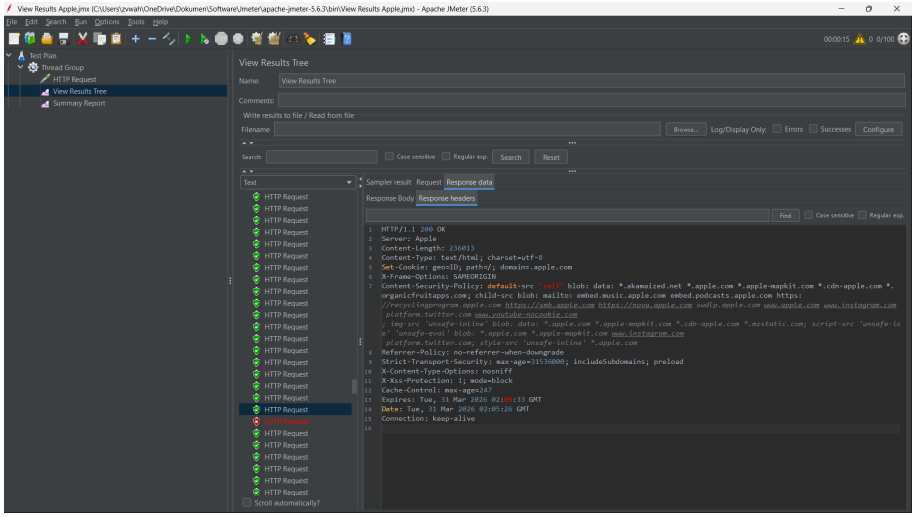
- **Number of Threads (users):** 100
- **Ramp-up period (seconds):** 10 (JMeter memasukkan 10 pengguna per detik)
- **Loop Count:** 5 (Tiap user mengulang skenario 5 kali)
- **Total Sample Requests:** 500 permintaan (100×5)

3.2.3 Hasil dan Analisis Performa

Berdasarkan data *Summary Report* dan *View Results Tree* yang direkam langsung oleh sistem selama *load testing*, diperoleh profil performa server apple.com sebagai berikut:



Gambar 8: Laporan Hasil Pengujian (Summary Report)



Gambar 9: Sampel *Response* Sukses pada View Results Tree

3.2.3.1 *Response Time*

- **Average (Rata-rata):** 1158 ms (1,15 detik)
- **Min (Tercepat):** 78 ms
- **Max (Terlama):** 8674 ms (8,67 detik)

Analisis: Secara keseluruhan, waktu respons website Apple sangat baik. Rata-rata pemuatan halaman adalah 1,15 detik (di bawah batas toleransi pemuatan standar 2-3 detik) dengan respons tercepat yang sangat impresif, yaitu 78 ms. Terdapat anomali dengan nilai *maksimal* mencapai 8,67 detik. Hal ini ditafsirkan sebagai wujud antrean antaramuka jaringan ketika titik *load* terpadat 100 *threads* masuk mengeksekusi permintaan beban puncak bersamaan.

3.2.3.2 *Throughput* dan Kecepatan Transaksi

- **Throughput:** 32,0 requests / second

Analisis: Selama durasi pengujian, server Apple konsisten mengakomodir dan melayani rata-rata 32 transaksi sukses per detik (*Requests per Second*). Fakta ini menegaskan daya dan keandalan (*reliability*) spesifikasi *server load balancing* Apple untuk menanggung volume keramaian simultan tanpa *drop* performa dan tanpa terjadinya *bottleneck*.

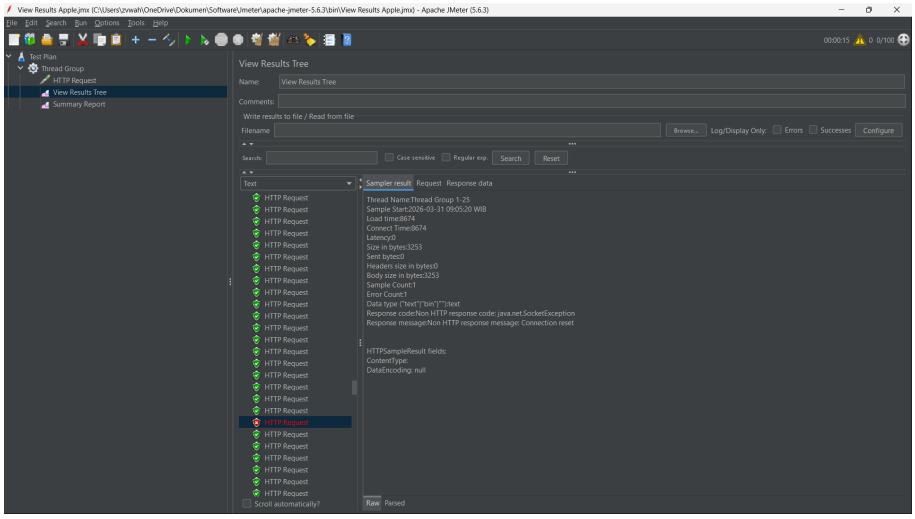
3.2.3.3 *Network Load* (Beban Jaringan)

- **Received (Diterima):** 7354,19 KB/sec (sekitar 7,3 MB/detik)
- **Sent (Dikirim):** 3,57 KB/sec

Analisis: Karena desain Apple sangat artistik dan memuat animasi atau *resource* gambar *High-Res*, aliran volume ukuran dokumen yang diterima klien dari server mencapai kisaran 7,3 MB per detik. Sebagai balasannya, nilai *sent* sangat wajar karena kecil, hanya mengedepankan HTTP GET.

3.2.3.4 *Error Rate* (Tingkat Kegagalan)

- **Error %:** 0,80%
- **Detail Error:** `java.net.SocketException: Connection reset`



Gambar 10: Sampel *Response* Gagal pada View Results Tree

Analisis: Proporsi kesalahan amat tipis di angka 0.80% (sekitar 4 gagal terkirim dari 500 sampel) tidak disebabkan oleh gangguan *server down* (Kode HTTP 500/503), melainkan pemutusan sepihak paksaan (*Connection Reset*). Indikasinya *Web Application Firewall (WAF)* milik Apple tanggap merespons lalu mereset koneksi curiga akibat serbuan beban frekuensi secara konstan dari pola akses robot/JMeter yang ditangkap sebagai kemungkinan *Denial of Service* skala mini.

Kesimpulan

Berdasarkan tugas yang telah dilakukan, dapat disimpulkan bahwa Apache JMeter merupakan tools *performance testing* yang powerful dan fleksibel. JMeter menyediakan fitur lengkap mulai dari dukungan multi-protokol, perekaman skenario, hingga pelaporan hasil yang komprehensif. Metrik-metrik seperti *Response Time*, *Throughput*, *TPS*, dan *Error Rate* sangat penting untuk mengevaluasi kualitas performa sebuah sistem secara objektif.

Dari studi kasus *Load Testing* pada halaman utama website Apple, server menunjukkan performa unggul dengan waktu respons rata-rata 1,15 detik dan angka memproses rata-rata 32 permintaan tiap detik (*Throughput*). Sistem keamanan seperti *Firewall/Load Balancer* juga beroperasi secara aktif mendeteksi dan menangani *spam/bot traffic* berlebih (0,80% tingkat kesalahan *Connection Reset*) tanpa harus membuat infrastruktur utama lumpuh atau kehilangan fungsi saat ditekan beban simulasi 100 *threads* berulang kali.

Daftar Pustaka

[1] Apache Software Foundation, *Apache JMeter User's Manual*. [Online]. Tersedia: `https://jmeter.apache.org/usermanual/index.html`. [Diakses: 31 Maret 2026].

[2] Apache Software Foundation, *Download Apache JMeter*. [Online]. Tersedia: `https://jmeter.apache.org/download_jmeter.cgi`. [Diakses: 31 Maret 2026].

[3] ISTQB, *Foundation Level Syllabus – Non-Functional Testing*. [Online]. Tersedia: `https://istqb.org`. [Diakses: 31 Maret 2026].